



22

Collecting User Input

In web applications, you can collect user input from standard HTML form elements that look and behave similarly on all browsers. With native UI platforms, collecting user input is more nuanced.

In this chapter, you'll learn how to work with the various React Native components that are used to collect user input. These include text input, selecting from a list of options, checkboxes, and date/time selectors. All of these are used in every app in cases of register or login flow, as well as the purchase form. The experience of creating such forms is very valuable and this chapter will help you to know how to create any form in your future apps. You'll learn the differences between iOS and Android and how to implement the appropriate abstractions for your app.

The following topics will be covered in this chapter:

- Collecting text input
- Selecting from a list of options
- Toggling between on and off
- Collecting date/time input

Technical requirements

You can find the code files for this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter22>.

Collecting text input

It turns out that there's a lot to think about when it comes to implementing text inputs. For example, should it have placeholder text? Is this sensitive data that shouldn't be displayed on the screen? Should you process text as it's entered or when the user moves to another field?

In web apps, there is a special `<input>` HTML element that allows you to collect user inputs. In React Native, for that purpose, we use the `TextInput` component. Let's build an example that renders several instances of the `<TextInput>` component:

```
function Input(props: InputProps) {
  return (
    <View style={styles.textInputContainer}>
      <Text style={styles.textInputLabel}>{props.label}</Text>
      <TextInput style={styles.textInput} {...props} />
    </View>
  );
}
```

We have implemented the `Input` component that we will reuse several times. Let's take a look at a few use cases of text inputs:

```
export default function CollectingTextInput() {
  const [changedText, setChangedText] = useState("");
  const [submittedText, setSubmittedText] = useState("");
  return (
    <View style={styles.container}>
      <Input label="Basic Text Input:" />
      <Input label="Password Input:" secureTextEntry />
      <Input label="Return Key:" returnKeyType="search" />
      <Input label="Placeholder Text:" placeholder="Search" />
      <Input
        label="Input Events:"
        onChangeText={(e) => {
          setChangedText(e);
        }}
        onSubmitEditing={(e) => {
          setSubmittedText(e.nativeEvent.text);
        }}
        onFocus={() => {
          setChangedText("");
          setSubmittedText("");
        }}
      />
      <Text>Changed: {changedText}</Text>
      <Text>Submitted: {submittedText}</Text>
    </View>
  );
}
```

```
);  
}
```

I won't go into depth about what each of these `<TextInput>` components is doing; there are labels in the `Input` components that explain this. Let's see what these components look like on the screen:

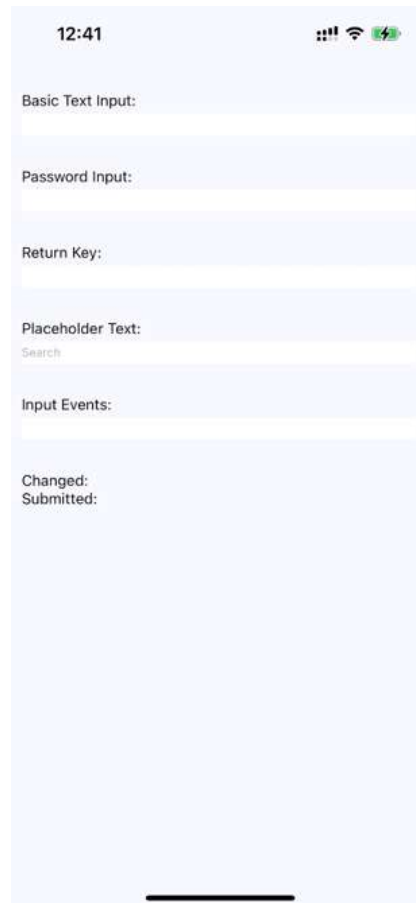


Figure 22.1: Text input variations

The plain text input shows the text that's been entered. The **Password Input** field doesn't reveal any characters. **Placeholder Text** is displayed when the input is empty. The **Changed** text state is also displayed. You can't see the **Submitted** text

state because I didn't press the **Submitted** button on the virtual keyboard before I took the screenshot.

Let's take a look at the virtual keyboard for the input element where you changed the **Return Key** text via the `returnKeyType` prop:

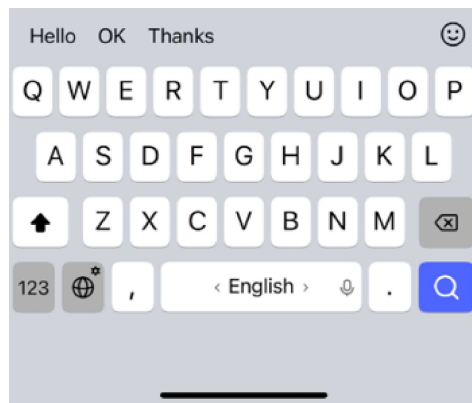


Figure 22.2: Keyboard with changed Return key text

When the keyboard **Return key** reflects what's going to happen when the user presses it, the user feels more in tune with the application.

One more common use case is changing the keyboard type. By providing the `keyboardType` prop to the `TextInput` component, you will see different variations of keyboards. This is convenient when you need to enter a PIN code or email address. Here is an example of a `numeric` keyboard:

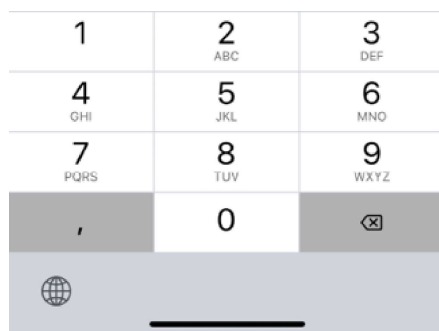


Figure 22.3: Numeric keyboard type

Now that you're familiar with collecting text input, it's time to learn how to select a value from a list of options.

Selecting from a list of options

In web applications, you typically use the `<select>` element to let the user choose from a list of options. React Native comes with a `Picker` component, which works on both iOS and Android, but in terms of reducing the React Native app size, the Meta team decided to delete it in future releases and extract `Picker` to its own package. To use that package, firstly, we install it in a clean project by running this command:

```
npx expo install @react-native-picker/picker
```

There is some trickery involved with styling this component based on which platform the user is on, so let's hide all of this inside a generic `Select` component. Here's the `Select.ios.js` module:

```
export default function Select(props: SelectProps) {
  return (
    <View style={styles.pickerHeight}>
      <View style={styles.pickerContainer}>
        <Text style={styles.pickerLabel}>{props.label}</Text>
        <Picker style={styles.picker} {...props}>
          {props.items.map((i) => (
            <Picker.Item key={i.label} {...i} />
          ))}
        </Picker>
      </View>
    </View>
  );
}
```

That's a lot of overhead for a simple `Select` component. It turns out that it's quite hard to style the React Native `Picker` component, because it looks completely different on iOS and Android. Despite that, we want to make it more cross-platform.

Here's the `Select.android.js` module:

```
export default function Select(props: SelectProps) {
  return (
```

```

<View>
  <Text style={styles.pickerLabel}>{props.label}</Text>
  <Picker {...props}>
    {props.items.map((i) => (
      <Picker.Item key={i.label} {...i} />
    ))}
  </Picker>
</View>
);
}

```

This is what the styles look like:

```

container: {
  flex: 1,
  flexDirection: "column",
  backgroundColor: "ghostwhite",
  justifyContent: "center",
},
pickersBlock: {
  flex: 2,
  flexDirection: "row",
  justifyContent: "space-around",
  alignItems: "center",
},
pickerHeight: {
  height: 250,
},

```

As usual with the `container` and `pickersBlock` styles, we define the base layout of the screen. Next, let's take a look at the styles of the `Select` component:

```

pickerContainer: {
  flex: 1,
  flexDirection: "column",
  alignItems: "center",
  backgroundColor: "white",
  padding: 6,
  height: 240,
},
pickerLabel: {
  fontSize: 14,
  fontWeight: "bold",
}

```

```

    },
    picker: {
      width: 150,
      backgroundColor: "white",
    },
    selection: {
      flex: 1,
      textAlign: "center",
    },
  },

```

Now, you can render your `Select` component. Here is what the `App.js` file looks like:

```

const sizes = [
  { label: "", value: null },
  { label: "S", value: "S" },
  { label: "M", value: "M" },
  { label: "L", value: "L" },
  { label: "XL", value: "XL" },
];

const garments = [
  { label: "", value: null, sizes: ["S", "M", "L", "XL"] },
  { label: "Socks", value: 1, sizes: ["S", "L"] },
  { label: "Shirt", value: 2, sizes: ["M", "XL"] },
  { label: "Pants", value: 3, sizes: ["S", "L"] },
  { label: "Hat", value: 4, sizes: ["M", "XL"] },
];

```

Here, we defined the default values for our `Select` component. Let's take a look at the final `SelectingOptions` component:

```

export default function SelectingOptions() {
  const [availableGarments, setAvailableGarments] = useState<typeof garments>(
    []
  );
  const [selectedSize, setSelectedSize] = useState<string | null>(null);
  const [selectedGarment, setSelectedGarment] = useState<number | null>(null);

```

With these hooks, we've implemented states of selectors. Next, we will use and pass them into components:

```
<View style={styles.container}>
  <View style={styles.pickersBlock}>
    <Select
      label="Size"
      items={sizes}
      selectedValue={selectedSize}
      onChange={(size: string) => {
        setSelectedSize(size);
        setSelectedGarment(null);
        setAvailableGarments(
          garments.filter((i) => i.sizes.includes(size))
        );
      }}
    />
    <Select
      label="Garment"
      items={availableGarments}
      selectedValue={selectedGarment}
      onChange={(garment: number) => {
        setSelectedGarment(garment);
      }}
    />
  </View>
  <Text style={styles.selection}>{selectedSize} && {selectedGarment} && `${selectedSize} ${garments.find((i) => i.value === selectedGarment)}
</View>
```

The basic idea of this example is that the selected option in the first selector changes the available options in the second selector. When the second selector changes, the label shows `selectedSize` and `selectedGarment` as a string. Here's how the screen looks:



Figure 22.4: Selecting from the list of options

The **Size** selector is shown on the left-hand side of the screen. When the **Size** value changes, the available values in the **Garment** selector on the right-hand side of the screen change to reflect size availability. The current selection is displayed as a string after the two selectors.

This is how our `app` looks on an Android device:

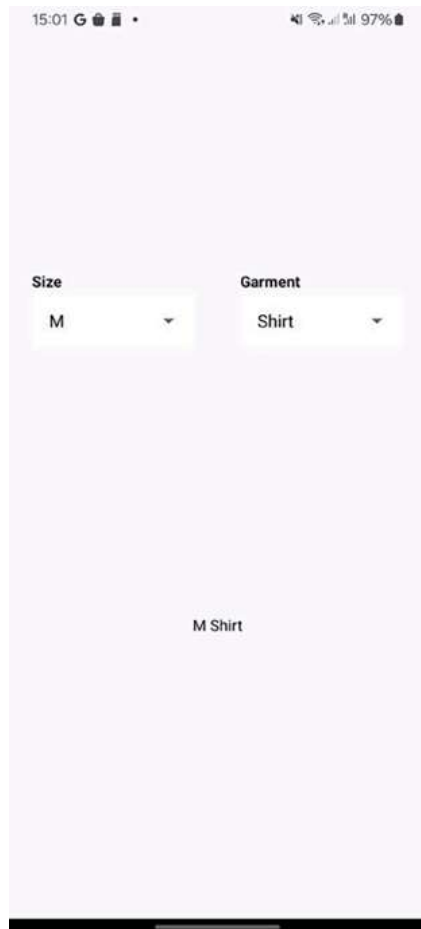


Figure 22.5: Selecting from the list of options on Android

When the iOS version of the `Picker` component renders a scrollable list of options, the Android one gives only buttons that open a dialog modal for selecting options.

In the following section, you'll learn about the buttons that toggle between on and off states.

Toggling between on and off

Another common element you'll see in web forms is checkboxes. For example, think of toggling Wi-Fi or Bluetooth on your device. React Native has a `Switch`

component that works on both iOS and Android. Thankfully, this component is a little easier to style than the `Picker` component. Let's look at a simple abstraction you can implement to provide labels for your switches:

```
type CustomSwitchProps = SwitchProps & {
  label: string;
};
export default function CustomSwitch(props: CustomSwitchProps) {
  return (
    <View style={styles.customSwitch}>
      <Text>{props.label}</Text>
      <Switch {...props} />
    </View>
  );
}
```

Now, let's learn how we can use a couple of switches to control application state:

```
export default function TogglingOnAndOff() {
  const [first, setFirst] = useState(false);
  const [second, setSecond] = useState(false);
  return (
    <View style={styles.container}>
      <Switch
        label="Disable Next Switch"
        value={first}
        disabled={second}
        onChange={setFirst}
      />
      <Switch
        label="Disable Previous Switch"
        value={second}
        disabled={first}
        onChange={setSecond}
      />
    </View>
  );
}
```

These two switches toggle the `disabled` property of one another. When the first switch is toggled, the `setFirst()` function is called, which will update the value of the first state. Depending on the current value of `first`, it will either be set to

`true` or `false`. The second switch works the same way, except it uses `setSecond()` and the second state value.

Turning on one switch will disable the other because we've set the `disabled` property value for each switch to the state of the other switch. For example, the second switch has `disabled={first}`, which means that it is disabled whenever the first switch is turned on. Here's what the screen looks like on iOS:

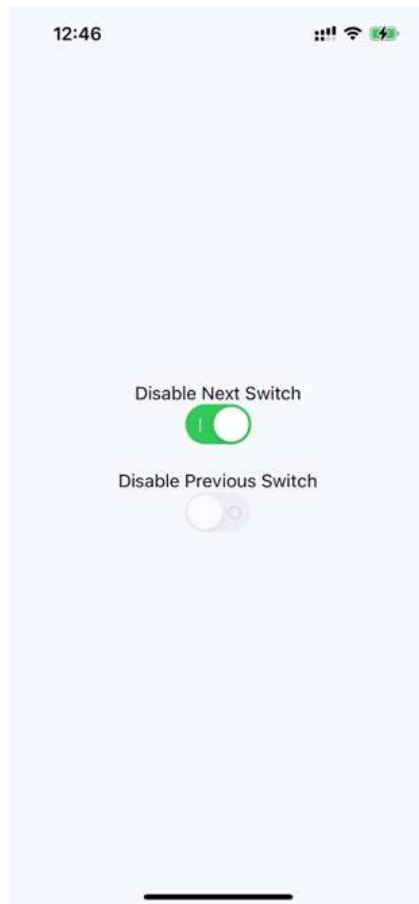


Figure 22.6: Switch toggles on iOS

Here's what the same screen looks like on Android:

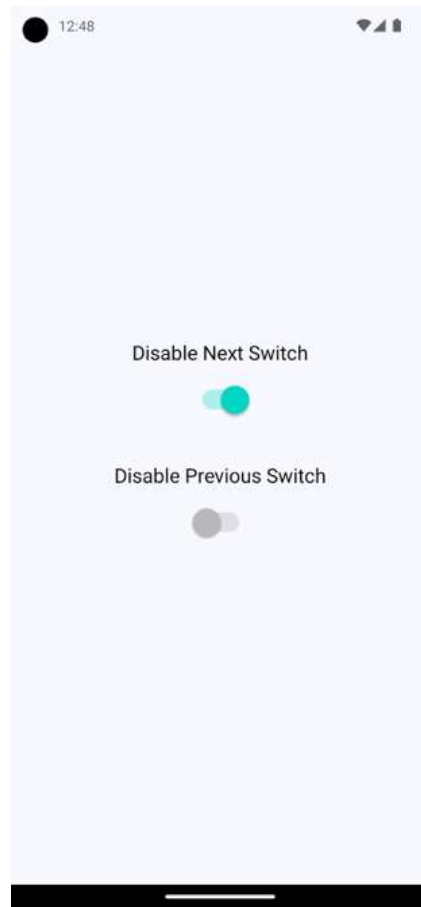


Figure 22.7: Switch toggles on Android

As you can see, our `CustomSwitch` component enables the same functionality on Android and iOS while using one component for both platforms. In the following section, you'll learn how to collect date/time input.

Collecting date/time input

In this final section of this chapter, you'll learn how to implement date/time pickers. React Native docs suggest using `@react-native-community/datetimepicker` independent date/time picker components for iOS and Android, which means that it is up to you to handle the cross-platform differences between the components.

To install `datetimepicker`, run the following command in the project:

```
npx expo install @react-native-community/datetimepicker
```

So, let's start with a `DatePicker` component for iOS:

```
export default function DatePicker(props: DatePickerProps) {
  return (
    <View style={styles.datePickerContainer}>
      <Text style={styles.datePickerLabel}>{props.label}</Text>
      <DateTimePicker
        mode="date"
        display="spinner"
        value={props.value}
        onChange={(event, date) => {
          if (date) {
            props.onChange(date);
          }
        }}
      />
    </View>
  );
}
```

There's not a lot to this component; it simply adds a label to the `DateTimePicker` component. The Android version works a bit differently; the better approach would be to use an **imperative API**. Let's take a look at the implementation:

```
export default function DatePicker({label, value, onChange }: DatePickerProps) {
  return (
    <View style={styles.datePickerContainer}>
      <Text style={styles.datePickerLabel}>{label}</Text>
      <Text
        onPress={() => {
          DateTimePickerAndroid.open({
            value: value,
            mode: "date",
            onChange: (event, date) => {
              if (event.type === "set" && date) {
                onChange(date);
              }
            },
          });
        }}
      />
    </View>
  );
}
```

```

    >
    {value.toLocaleDateString()}
  </Text>
</View>
);
}

```

The key difference between the two date pickers is that the Android version doesn't use a React Native component like `DateTimePicker` in iOS. Instead, we have to use the imperative `DateTimePickerAndroid.open()` API. This is triggered when the user presses the date text that our component renders and opens a date picker dialog. The good news is that this component of ours hides this API behind a declarative component.



I've also implemented a time picker component that follows this exact pattern. So, rather than listing that code here, I suggest that you download the code for this book from <https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter22> so that you can see the subtle differences and run the example.

Now, let's learn how to use our date and time picker components:

```

export default function CollectingDateTimeInput() {
  const [date, setDate] = useState(new Date());
  const [time, setTime] = useState(new Date());
  return (
    <View style={styles.container}>
      <DatePicker
        label="Pick a date, any date:"
        value={date}
        onChange={setDate}
      />
      <TimePicker
        label="Pick a time, any time:"
        value={time}
        onChange={setTime}
      />
    </View>
  );
}

```

Awesome! Now, we have `DatePicker` and `TimePicker` components that can help us select dates and times in our app. Also, they both work on iOS and Android. Let's see how the pickers look on iOS:

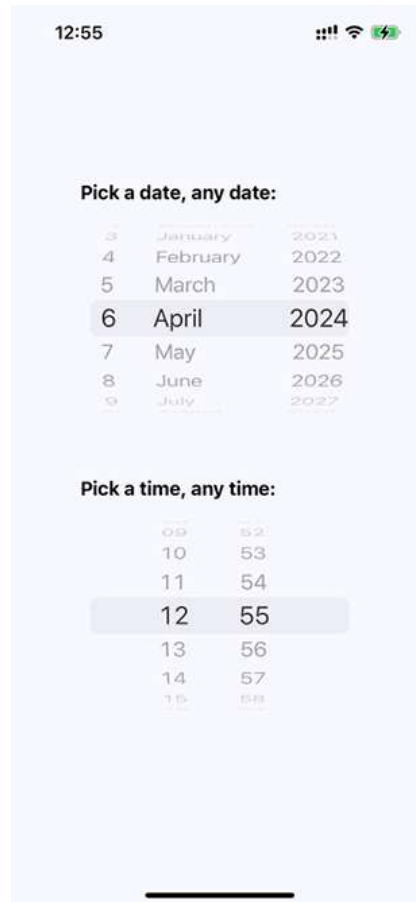


Figure 22.8: iOS date and time pickers

As you can see, the iOS date and time pickers use the `Picker` component that you learned about earlier in this chapter. The Android picker looks a lot different; let's look at it now:

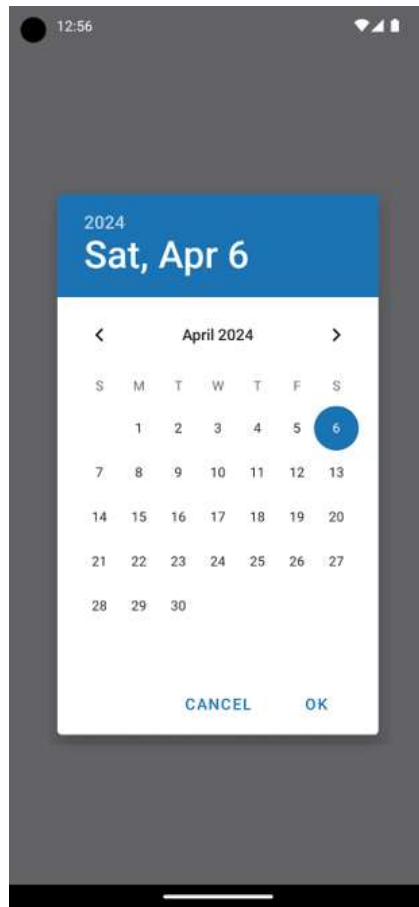


Figure 22.9: Android date picker

The Android version follows a completely different approach from the iOS date/time picker, yet we can use the same `DatePicker` component that we've created on both platforms. This brings us to the end of the chapter.

Summary

In this chapter, we learned about the various React Native components that resemble the form elements from the web that we're used to. We started off by learning about text input and how each text input has its own virtual keyboard to take into consideration. Next, we learned about `Picker` components, which allow the user to select an item from a list of options. Then, we learned about the

`Switch` component, which is kind of like a checkbox. With these components, you will be able to build a form of any complexity.

In the final section, we learned how to implement generic date/time pickers that work on both iOS and Android. In the next chapter, we'll learn about modal dialogs in React Native.